

Practico 5

Generación de variables aleatorias continuas

Índice:

1. [Ejercicio 1 - pág. 2](#)
 2. [Ejercicio 2 - pág. 7](#)
 3. [Ejercicio 3 - pág. 11](#)
 4. [Ejercicio 4 - pág. 14](#)
 5. [Ejercicio 5 - pág. 16](#)
 6. [Ejercicio 6 - pág. 18](#)
 7. [Ejercicio 7 - pág. 22](#)
 8. [Ejercicio 8 - pág. 25](#)
 9. [Ejercicio 9](#)
 10. [Ejercicio 10](#)
 11. [Ejercicio 11](#)
 12. [Ejercicio 12](#)
 13. [Ejercicio 13](#)
 14. [Ejercicio 14](#)
-

Ejercicio 1 ([Índice](#))

Enunciado: Desarrolle un método para generar una variable aleatoria cuya densidad de probabilidad es:

- a)

$$f(x) = \begin{cases} \frac{x-2}{2} & \text{si } 2 \leq x \leq 3 \\ \frac{2-x/3}{2} & \text{si } 3 \leq x \leq 6 \\ 0 & \text{en otro caso} \end{cases}$$

- b)

$$f(x) = \begin{cases} \frac{6(x+3)}{35} & \text{si } 0 \leq x \leq 1 \\ \frac{6x^2}{35} & \text{si } 1 \leq x \leq 2 \\ 0 & \text{en otro caso} \end{cases}$$

- c)

$$f(x) = \begin{cases} \frac{\exp(4x)}{4} & \text{si } -\infty < x \leq 0 \\ \frac{1}{4} & \text{si } 0 < x \leq \frac{15}{4} \\ 0 & \text{en otro caso} \end{cases}$$

Solución:

- a)
 - Necesitamos calcular $F(x)$:
 - Para $2 \leq x \leq 3$:

$$F(x) = \int_2^x \frac{t-2}{2} dt = \frac{1}{2} \left[\frac{(t-2)^2}{2} \right]_2^x = \frac{(x-2)^2}{4}$$

- Para $3 \leq x \leq 6$:

$$\begin{aligned} F(x) &= F(3) + \int_3^x \frac{2-t/3}{2} dt = \frac{1}{4} + \frac{1}{2} \left[2t - \frac{t^2}{6} \right]_3^x \\ &= \frac{1}{4} + \frac{1}{2} \left(2x - \frac{x^2}{6} - \left(6 - \frac{9}{6} \right) \right) \\ &= \frac{1}{4} + \frac{1}{2} \left(2x - \frac{x^2}{6} - \frac{27}{6} \right) \\ &= \frac{1}{4} + \frac{1}{2} \left(2x - \frac{x^2}{6} - \frac{9}{2} \right) \\ &= \frac{1}{4} + x - \frac{x^2}{12} - \frac{9}{4} \\ &= x - \frac{x^2}{12} - 2 \end{aligned}$$

- Verificando:

$$F(6) = 6 - \frac{36}{12} - 2 = 6 - 3 - 2 = 1$$

- Ahora realizamos la inversa:
- Para $0 \leq u \leq 1/4$:

$$u = \frac{(x-2)^2}{4} \implies x = 2 + 2\sqrt{u}$$

- Para $1/4 \leq u \leq 1$:

$$\begin{aligned} u &= x - \frac{x^2}{12} - 2 \\ -12u &= -12x + x^2 + 12 \\ 0 &= x^2 - 12x + 12u + 24 \end{aligned}$$

- Tomando $a : 1$, $b : -12$ y $c : 12u + 24$,

$$x = \frac{12 \pm \sqrt{144 - 4(12u + 24)}}{2} = 6 \pm \sqrt{36 + 12u + 24} = 6 \pm \sqrt{12 - 12u}$$

- Para $x \in [3, 6]$, tomamos la rama negativa:

$$x = 6 - \sqrt{12(1-u)} = 6 - 2\sqrt{3(1-u)}$$

- Y de esta manera tenemos el siguiente algoritmo:



```

1  import math
2  from random import random
3
4  def densidad_a():
5      U = random()
6      if U < 0.25:
7          return 2 + 2 * math.sqrt(U)
8      else:
9          return 6 - 2 * math.sqrt(3 * (1 - U))

```

- b)
- Necesitamos calcular $F(x)$:

- Para $0 \leq x \leq 1$:

$$F(x) = \int_0^x \frac{6(t+3)}{35} dt = \frac{6}{35} \left[\frac{t^2}{2} + 3t \right]_0^x = \frac{6}{35} \left(\frac{x^2}{2} + 3x \right) = \frac{3x^2 + 18x}{35}$$

- Para $1 \leq x \leq 2$:

$$\begin{aligned} F(x) &= F(1) + \int_1^x \frac{6t^2}{35} dt \\ &= \frac{3}{5} + \frac{6}{36} \cdot \left[\frac{t^3}{3} \right]_1^x \\ &= \frac{3}{5} + \frac{2}{35} (x^3 - 1) \\ &= \frac{3}{5} + \frac{2x^3}{35} - \frac{2}{35} \\ &= \frac{21}{35} + \frac{2x^3}{35} - \frac{2}{35} \\ &= \frac{2x^3 + 19}{35} \end{aligned}$$

- Ahora realizamos la inversa:

- Para $0 \leq u \leq 3/5$:

$$u = \frac{3x^2 + 18x}{35} \implies 3x^2 + 18x - 35u = 0$$

- $$x = \frac{-18 + \sqrt{324 + 420u}}{6} = \frac{-18 + \sqrt{36(9 + 35u/3)}}{6} = \frac{-18 + 6\sqrt{9 + 35u/3}}{6}$$

- $$x = -3 + \sqrt{9 + \frac{35u}{3}}$$

- Para $3/5 \leq u \leq 1$:

$$u = \frac{2x^3 + 19}{35} \implies 2x^3 = 35u - 19 \implies x^3 = \frac{35u - 19}{2}$$

- $$x = \left(\frac{35u - 19}{2} \right)^{1/3}$$

- Y de esta manera tenemos el siguiente algoritmo:

```

1 import math
2 from random import random
3
4 def densidad_b():
5     U = random()
6     if U < 0.6:
7         return -3 + math.sqrt(9 + 35 * U / 3)
8     else:
9         return ((35 * U - 19) / 2) ** (1/3)

```

• c)

- Necesitamos calcular $F(x)$:
- Para $x \leq 0$:

$$F(x) = \int_{-\infty}^x \frac{e^{4t}}{4} dt = \frac{3^{4x}}{16}$$

- Para $0 < x \leq 15/4$:

$$\begin{aligned}
 F(x) &= F(0) + \int_0^x \frac{1}{4} dt \\
 &= \frac{1}{16} + \int_0^x \frac{1}{4} dt \\
 &= \frac{1}{16} + \frac{x}{4}
 \end{aligned}$$

- Ahora realizamos la inversa:
- Para $0 \leq u \leq 1/16$:

$$u = \frac{e^{4x}}{16} \implies e^{4x} = 16u \implies x = \frac{1}{4} \ln(16u)$$

- Para $1/16 \leq u \leq 1$:

$$u = \frac{1}{16} + \frac{x}{4} \implies \frac{x}{4} = u - \frac{1}{16} \implies x = 4u - \frac{1}{4}$$

- Y de esta manera tenemos el siguiente algoritmo:



```
1 import math
2 from random import random
3
4 def densidad_c():
5     U = random()
6     if U < 1/16:
7         return 0.25 * math.sqrt(16 * U)
8     else:
9         return 4 * U - 0.25
```

•

Ejercicio 2 ([Índice](#))

Enunciado:

- a) Desarrolle métodos para generar las siguientes variables aleatorias
 - i) Distribución Pareto

$$f(x) = ax^{-(a+1)} \quad 1 \leq x < \infty, \quad a > 0$$

- ii) Distribución Erlang

$$f(x) = \frac{x^{k-1} \exp(-x/\mu)}{(k-1)!\mu^k} \quad 0 \leq x < \infty, \quad \mu > 0, \quad k \text{ entero}$$

- iii) Distribución Weibull

$$f(x) = \frac{\beta}{\lambda} \left(\frac{x}{\lambda}\right)^{\beta-1} \exp(-(x/\lambda)^\beta) \quad x \geq 0, \quad \lambda > 0, \quad \beta > 0$$

Ayuda: La distribución Pareto y la distribución Weibull tienen distribución acumulada F con forma cerrada, por lo cual puede aplicarse el método de la transformada Inversa. La distribución de Erlang pertenece a la familia de las Gammas. Puede simularse por rechazo o como suma de exponenciales.

- b) Estime la media de cada variable con 10.000 repeticiones, usando parámetros $a = 2, \mu = 2, k = 2, \lambda = 1, \beta = 2$. Busque en la web los valores de las esperanzas para cada variable con estos parámetros(cuidado con las parametrizaciones) y compare los valores obtenidos.

Solución:

- a)
 - i) Distribución de Pareto:
 - Densidad:

$$f(x) = ax^{-(a+1)} \quad 1 \leq x < \infty, \quad a > 0$$

- Función de distribución acumulada:

$$F(x) = \int_1^x at^{-(a+1)} dt = 1 - x^{-a}, \quad x \geq 1$$

- Para aplicar el método de la transformada inversa:

$$U = F(X) = 1 - X^{-a} \implies X^{-a} = 1 - U \implies X = (1 - U)^{-1/a}$$

- Como $1 - U \sim \text{Uniforme}(0, 1)$, también podemos escribir:

$$X = U^{-1/a}$$

- Y de esta manera, tenemos el siguiente algoritmo:



```
1 from random import random
2
3 def pareto(a):
4     U = random()
5     return U ** (-1/a)
```

- ii) Distribución Erlang

- Densidad:

$$f(x) = \frac{x^{k-1} \exp(-x/\mu)}{(k-1)! \mu^k} \quad x \geq 0, \quad \mu > 0, \quad k \text{ entero}$$

- La distribución Erlang es un caso especial de la distribución Gamma:
 $X \sim \text{Gamma}(k, 1/\mu)$ con **forma entera**. Por la propiedad de la suma de exponenciales:

$$X = \sum_{i=1}^k E_i, \quad E_i \sim \text{Exp}(\lambda = 1/\mu) \text{ independientes}$$

- Cada E_i se genera por transformada inversa: $E_i = -\mu \ln(1 - U_i)$.
- Ya que en el apunte tenemos un algoritmo que genera exponenciales con un parámetro lambda, tenemos el siguiente algoritmo:



```
1 from transf_inversa import continuousGenerators as cg
2
3 def erlang(k, mu):
4     lam = 1 / mu
5     suma = 0
6     for _ in range(k):
7         suma += cg.exponentialL(lam)
8     return suma
```

- iii) Distribución Weibull

- Densidad:

$$f(x) = \frac{\beta}{\lambda} \left(\frac{x}{\lambda} \right)^{\beta-1} \exp(-(x/\lambda)^\beta) \quad x \geq 0, \quad \lambda > 0, \quad \beta > 0$$

- Función de distribución acumulada:

$$F(x) = 1 - e^{-(x/\lambda)^\beta}$$

- Método de transformada inversa:

$$\begin{aligned} U &= 1 - e^{-(X/\lambda)^\beta} \implies e^{-(X/\lambda)^\beta} = 1 - U \\ -(X/\lambda)^\beta &= \ln(1 - U) \implies (X/\lambda)^\beta = -\ln(1 - U) \\ X &= \lambda(-\ln(1 - U))^{1/\beta} \end{aligned}$$

- Como $1 - U \sim \text{Uniforme}(0, 1)$:

$$X = \lambda(-\ln(U))^{1/\beta}$$

- Y de esta manera tenemos el siguiente algoritmo:



```

1  from random import random
2  import math
3
4  def weibull(lam, beta):
5      U = random()
6      return lam * (-math.log(U)) ** (1/beta)

```

- b)

- Distribución Pareto

- Parámetros $a = 2$
- Esperanza teórica:

$$E[X] = \frac{a}{a-1} = \frac{2}{1} = 2 \quad (\text{para } x \geq 1)$$

- Distribución Erlang

- Parámetros $k = 2, \mu = 2$
- Esperanza teórica:

$$E[X] = k\mu = 2 \times 2 = 4$$

- Distribución Weibull

- Parámetros $\lambda = 1, \beta = 2$

- Esperanza teórica:

$$E[X] = \lambda \Gamma(1 + 1/\beta) = 1 \times \Gamma(1.5) = \frac{\sqrt{\pi}}{2} \approx 0.8862$$

- Mediante el siguiente algoritmo obtenemos las estimaciones:

```

1  def estimacion():
2      n = 10000
3
4      sum_pareto = 0
5      sum_erlang = 0
6      sum_weibull = 0
7
8      for _ in range(n):
9          sum_pareto += pareto(2)
10         sum_erlang += erlang(2, 2)
11         sum_weibull += weibull(1, 2)
12
13     media_pareto = sum_pareto / n
14     media_erlang = sum_erlang / n
15     media_weibull = sum_weibull / n
16
17     print(f"Pareto: {media_pareto:.4f}")
18     print(f"Erlang: {media_erlang:.4f}")
19     print(f"Weibull: {media_weibull:.4f}")

```

-
- Obteniendo así los siguientes resultados:

Distribución	Estimada	Teórica
Pareto	1.9801	2
Erlang	3.9819	4
Weibull	0.8816	0.8862

Ejercicio 3 ([Índice](#))

Enunciado: Método de la composición:

- a) Suponga que es relativamente fácil generar n variables aleatorias a partir de sus distribuciones de probabilidad F_i , $i = 1, \dots, n$. Implemente un método para generar una variable aleatoria cuya distribución de probabilidad es

$$F(x) = \sum_{i=1}^n p_i F_i(x)$$

donde p_i , $i = 1, \dots, n$, son números no negativos cuya suma es 1.

- b) Genere datos usando tres exponenciales independientes con media 3, 5 y 7 respectivamente y $p = (0.5, 0.3, 0.2)$. Calcule la esperanza exacta de la mezcla y estime con 10.000 repeticiones. Tenga cuidado con la parametrización que este usando!!

Solución:

- a) Dado:

$$F(x) = \sum_{i=1}^n p_i F_i(x), \quad \sum_{i=1}^n p_i = 1, \quad p_i \geq 0$$

- El método de composición dice:
 1. Generar $U \sim \text{Uniforme}(0, 1)$.
 2. Elegir un índice i con probabilidad p_i .
 3. Generar X a partir de la distribución F_i .
 4. Devolver X .
- De esta manera, tenemos el siguiente algoritmo:

```

1  from random import random
2
3  def ejercicio3a(p_list, gen_list):
4      """
5      p_list: Lista de probabilidades
6      gen_list: lista de funciones que
7                  generan una variable
8                  de la distribución F_i
9      """
10     U = random()
11     acum = 0
12     for i, p in enumerate(p_list):
13         acum += p
14         if U < acum:
15             return gen_list[i]()
16     return gen_list[-1]()

```

- b)

- Datos

- Exp_1 : media 3 $\rightarrow$ parámetro $\lambda_1 = 1/3$
- Exp_2 : media 5 $\rightarrow$ parámetro $\lambda_2 = 1/5$
- Exp_3 : media 7 $\rightarrow$ parámetro $\lambda_3 = 1/7$
- Probabilidades: $p = (0.5, 0.3, 0.2)$

- Esperanza exacta de la mezcla:

$$\begin{aligned}
 E[X] &= \sum_{i=1}^n p_i \cdot E[X_i] = 0.5 \times 3 + 0.3 \times 5 + 0.2 \times 7 \\
 &= 1.5 + 1.5 + 1.4 \\
 &= 4.4
 \end{aligned}$$

- Mediante el siguiente algoritmo, utilizando el método de generación de exponenciales del apunte, estimamos:

```
1 from random import random
2 from transf_inversa import continuousGenerators as cg
3
4 def ejercicio3b():
5     p = [0.5, 0.3, 0.2]
6     medias = [3, 5, 7]
7     lambdas = [1/mu for mu in medias]
8
9     U = random()
10    acum = 0
11    for i, prob in enumerate(p):
12        acum += prob
13        if U < acum:
14            return cg.exponentialL(lambdas[i])
15
16 def simulate():
17     n = 10000
18     suma = 0
19     for _ in range(n):
20         suma += ejercicio3b()
21
22     media_estimada = suma / n
23
24     print(f"Esperanza exacta: 4.4")
25     print(f"Media estiamda con {n} repeticiones: {media_estimada:.4f}")
```

Ejercicio 4 ([Índice](#))

Enunciado: Desarrolle un método para generar la variable aleatoria con función de distribución

$$F(x) = \int_0^{\infty} x^y e^{-y} dy, \quad 0 \leq x \leq 1.$$

Piense en el método de composición del ejercicio anterior. En particular, sea F la función de distribución de X y suponga que la distribución condicional de X dado $Y = y$ es

$$P(X \leq x | Y = y) = x^y, \quad 0 \leq x \leq 1.$$

Solución:

- Identificación de la distribución marginal de Y
 - Si

$$F(x) = \int_0^{\infty} F_{X|Y}(x|y) f_Y(y) dy$$

comparando con la expresión dada:

$$F(x) = \int_0^{\infty} x^y e^{-y} dy$$

se deduce que:

$$f_Y(y) = e^{-y}, \quad y > 0$$

es decir, $Y \sim \text{Exp}(1)$ (exponencial con media 1, parámetro $\lambda = 1$).

- Método de composición:
 1. Generar $Y \sim \text{Exp}(1)$.
 2. Dado $Y = y$, generar X con distribución $P(X \leq x | Y = y) = x^y$.
La generación de X condicional a $Y = y$ se puede hacer por **transformada inversa**:

$$U = X^y \implies X = U^{1/y}.$$

Si $U \sim \text{Uniforme}(0, 1)$, entonces $P(U^{1/y} \leq x) = P(U \leq x^y) = x^y$.

- Algoritmo completo
 1. Generar $U_1 \sim \text{Uniforme}(0, 1)$
 2. Calcular $Y = -\ln(1 - U_1)$ (método de transformada inversa para $\text{Exp}(1)$).
 3. Generar $U_2 \sim \text{Uniforme}(0, 1)$ independiente.
 4. Calcular $X = U_2^{1/Y}$.
 5. Devolver X .
- Tenemos el siguiente algoritmo:



```
1 from random import random
2 from transf_inversa import continuousGenerators as cg
3
4 def generar_X():
5     Y = cg.exponential1()
6
7     U = random()
8     X = U ** (1 / Y)
9     return X
```

•

Ejercicio 5 ([Índice](#))

Enunciado:

- a) Considere que es sencillo generar una variable aleatoria a partir de cualquiera de las distribuciones F_i , $i = 1, \dots, n$. Explique cómo generar variables aleatorias a partir de las siguientes distribuciones:

- i) $F_M(x) = \prod_{i=1}^n F_i(x)$
- ii) $F_m(x) = 1 - \prod_{i=1}^n (1 - F_i(x))$

Sugerencia: Si X_i , $i = 1, \dots, n$, son variables aleatorias independientes, donde X_i tiene distribución F_i , ¿Cuál variable tiene como distribución a F en cada caso?

- b) Genere una muestra de 10 valores de las variables M y m con distribuciones F_M y F_m si X_i son exponenciales independientes con parámetros 1, 2 y 3 respectivamente.

Solución:

Dadas $X_1, \dots, X_n$ independientes, con $X_i \sim F_i$.

- a) Identificación de F_M y F_m

- i) $F_M(x) = \prod_{i=1}^n F_i(x)$
 - Observamos que:

$$P(\max(X_1, \dots, X_n) \leq x) = P(X_1 \leq x, \dots, X_n \leq x) = \prod_{i=1}^n F_i(x)$$

- Por lo tanto:

$$F_M(x) = F_{\max}(x)$$

- Conclusión: $M = \max(X_1, \dots, X_n)$ tiene distribución F_M .

- Método de generación:

- Generar $X_1, \dots, X_n$ independientes con distribuciones F_i y devolver $M = \max(X_1, \dots, X_n)$.

- ii) $F_m(x) = 1 - \prod_{i=1}^n (1 - F_i(x))$
 - Observamos que:

$$1 - F_m(x) = \prod_{i=1}^n (1 - F_i(x))$$

- Pero $1 - F_i(x) = P(X_i > x)$. Luego:

$$\prod_{i=1}^n P(X_i > x) = P(X_1 > x, \dots, X_n > x) = P(\min(X_1, \dots, X_n) > x)$$

- Por lo tanto:

$$1 - F_m(x) = P(\min(X_1, \dots, X_n) > x) \implies F_m(x) = P(\min(X_1, \dots, X_n) \leq x)$$

- Conclusión: $m = \min(X_1, \dots, X_n)$ tiene distribución F_m .
- Método de generación:
 - Generar $X_1, \dots, X_n$ independientes con distribución F_i y devolver $m = \min(X_1, \dots, X_n)$.

• b)

- Datos:
 - $X_1 \sim \text{Exp}(\lambda_1 = 1) \rightarrow$, media 1, función de distribución $F_1(x) = 1 - e^{-x}$
 - $X_2 \sim \text{Exp}(\lambda_2 = 2) \rightarrow$, media 1/2, $F_2(x) = 1 - e^{-2x}$
 - $X_3 \sim \text{Exp}(\lambda_3 = 3) \rightarrow$, media 1/3, $F_3(x) = 1 - e^{-3x}$
- Mediante el siguiente algoritmo generamos las muestras:

```

1  from random import random
2  from transf_inversa import continuousGenerators as cg
3
4  def generar_max():
5      X1 = cg.exponentialL(1)
6      X2 = cg.exponentialL(2)
7      X3 = cg.exponentialL(3)
8      return max(X1, X2, X3)
9
10 def generar_min():
11     X1 = cg.exponentialL(1)
12     X2 = cg.exponentialL(2)
13     X3 = cg.exponentialL(3)
14     return min(X1, X2, X3)
15
16 def simular():
17     print(f"Muestra de M (máximo)")
18     for _ in range(10):
19         print(f"{generar_max():.4f}")
20
21     print(f"\nMuestra de m (mínimo)")
22     for _ in range(10):
23         print(f"{generar_min():.4f}")

```

Ejercicio 6 (Índice)

Enunciado: Utilice el método del rechazo y los resultados del ejercicio anterior para desarrollar otros dos métodos, además del método de la transformada inversa, para generar una variable aleatoria con distribución de probabilidad

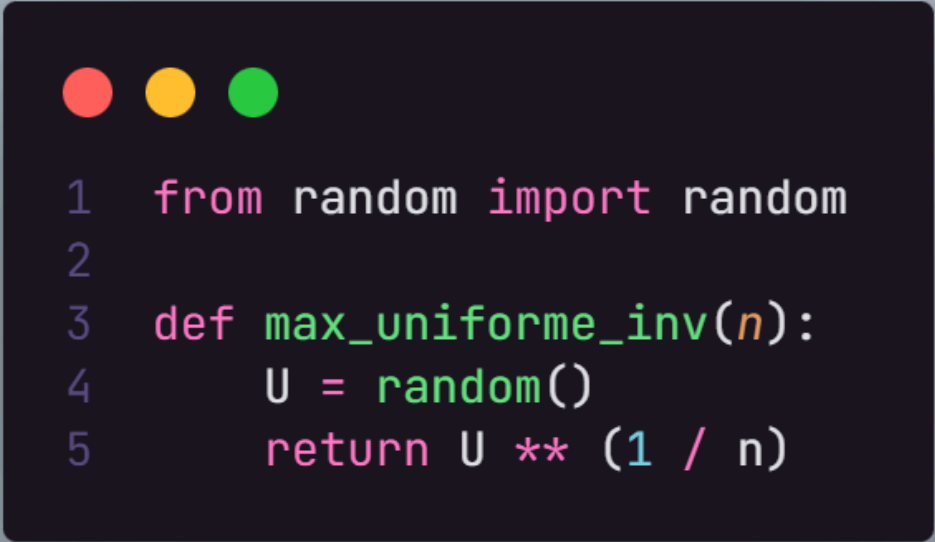
$$F(x) = x^n, \quad 0 \leq x \leq 1.$$

Analice la eficiencia de los tres métodos para generar la variable a partir de F .

Solución:

- Método 1: Transformada inversa

- $U = X^n \implies X = U^{1/n}$
- Sea el siguiente algoritmo:



```
1 from random import random
2
3 def max_uniforme_inv(n):
4     U = random()
5     return U ** (1 / n)
```

- Método 2: Por composición usando el máximo

- Del ejercicio 5, sabemos que:

$$F_M(x) = \prod_{i=1}^n F_i(x) \implies M = \max(X_1, \dots, X_n)$$

- Si tomamos $X_i \sim \text{Uniforme}(0, 1)$, entonces $F_i(x) = x$ para $0 \leq x \leq 1$.
- Por lo tanto:

$$F_M(x) = x^n$$

- Conclusión: Generar n uniformes y tomar el máximo es equivalente.
- Tenemos el siguiente algoritmo:



```
1  from random import random
2
3  def max_uniforme_composicion(n):
4      maximo = 0
5      for _ in range(n):
6          U = random()
7          if U > maximo:
8              maximo = U
9      return maximo
```

- Método 3: Por aceptación y rechazo
 - Necesitamos la densidad:

$$f(x) = nx^{n-1}, \quad 0 \leq x \leq 1$$

- Usamos una densidad candidata simple. Elegimos $g(x) = 1$ (uniforme en $[0, 1]$).
- La constante c debe satisfacer:

$$c \geq \frac{f(x)}{g(x)} = nx^{n-1}, \quad \forall x \in [0, 1]$$

- El máximo de nx^{n-1} en $[0, 1]$ ocurre en $x = 1$:

$$\max_{x \in [0, 1]} nx^{n-1} = n$$

- Tomamos $c = n$.
- Entonces:

$$\frac{f(x)}{c \cdot g(x)} = \frac{nx^{n-1}}{n \cdot 1} = x^{n-1}$$

- De esta manera, tenemos el siguiente algoritmo:



```
1  from random import random
2
3  def max_uniforme_rechazo(n):
4      while True:
5          Y = random()
6          U = random()
7          if U < Y ** (n-1):
8              return Y
```

- Finalmente analizamos la eficiencia de cada uno:



```
1  def eficiencia():
2      n = 10
3      repeticiones = 10000
4
5      start_inv = time.time()
6      for _ in range(repeticiones):
7          _ = max_uniforme_inv(n)
8      end_inv = time.time()
9      print(f"Inversa: {end_inv - start_inv:.4f} seg")
10
11     start_comp = time.time()
12     for _ in range(repeticiones):
13         _ = max_uniforme_composicion(n)
14     end_comp = time.time()
15     print(f"Composicion: {end_comp - start_comp:.4f} seg")
16
17     start_rech = time.time()
18     for _ in range(repeticiones):
19         _ = max_uniforme_rechazo(n)
20     end_rech = time.time()
21     print(f"Rechazo: {end_rech - start_rech:.4f} seg")
```

•

Método	Tiempo
Inversa	0.0014
Composición	0.0062
Rechazo	0.0126



Ejercicio 7 ([Índice](#))

Enunciado:

- a) Desarrolle dos métodos para generar una variable aleatoria X con densidad de probabilidad:

$$f(x) = \begin{cases} \frac{1}{x} & \text{si } 1 \leq x \leq e \\ 0 & \text{en otro caso} \end{cases}$$

- i) Aplicando Transformada inversa.
- ii) Aplicando el método de aceptación y rechazo con una variable uniforme.
- b) Compare la eficiencia de ambos métodos realizando 10.000 simulaciones y comparando el promedio de los valores obtenidos. Compruebe que se obtiene un valor aproximado del valor esperando de X .
- c) Estime la probabilidad $P(X \leq 2)$ y compárela con el valor real.

Solución:

- a) Métodos para generar X con $f(x) = 1/x$, $1 \leq x \leq e$
 - i) Método de la transformada inversa
 - Primero calculamos la función de distribución acumulada:

$$F(x) = \int_1^x \frac{1}{t} dt = \ln t \Big|_1^x = \ln x, \quad 1 \leq x \leq e$$

- Aplicamos transformada inversa:

$$U = F(X) = \ln X \implies X = e^U$$

donde $U \sim \text{Uniforme}(0, 1)$.

- Sea el siguiente algoritmo:
- ii) Método de aceptación y rechazo con una variable uniforme
 - Elegimos como variable candidata $Y \sim \text{Uniforme}(1, e)$
 - Su densidad es $g(y) = \frac{1}{e-1}$ para $1 \leq y \leq e$.
 - Buscamos una constante c tal que:

$$\frac{f(y)}{g(y)} = \frac{1/y}{1/(e-1)} = \frac{e-1}{y} \leq c, \quad \forall y \in [1, e]$$

- El cociente $\frac{e-1}{y}$ es máximo cuando y es mínimo, es decir, $y = 1$:

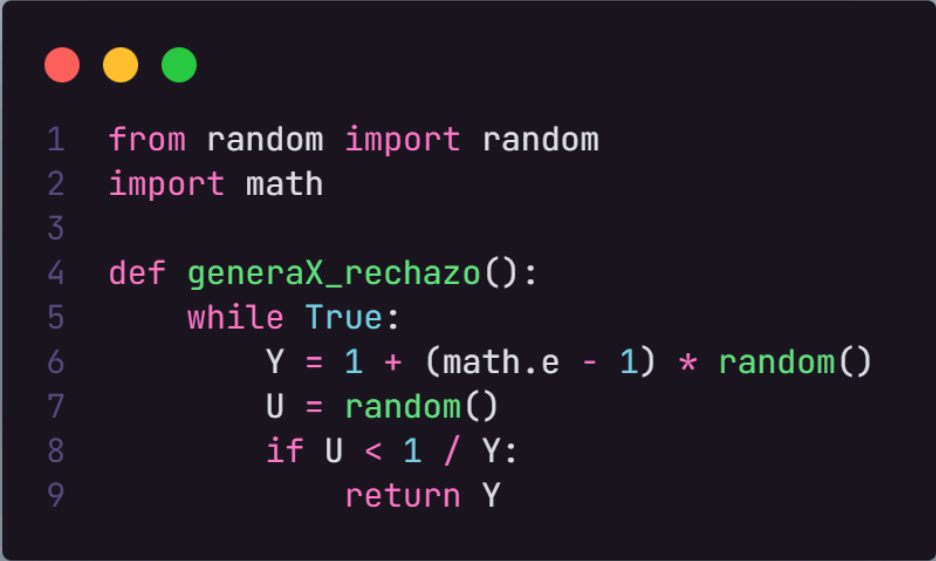
$$\max_{y \in [1, e]} \frac{e-1}{y} = e-1$$

- Tomamos $c = e-1 \approx 1.71828$

- Luego:

$$\frac{f(y)}{c \cdot g(y)} = \frac{1/y}{(e-1) \cdot \frac{1}{e-1}} = \frac{1/y}{1} = \frac{1}{y}$$

- Porque $c \cdot g(y) = (e-1) \cdot \frac{1}{e-1} = 1$
- Sea el siguiente algoritmo:



```

1  from random import random
2  import math
3
4  def generaX_rechazo():
5      while True:
6          Y = 1 + (math.e - 1) * random()
7          U = random()
8          if U < 1 / Y:
9              return Y

```

- b) Comparación de eficiencia
 - Valor exacto:

$$E[X] = \int_1^e x \left(\frac{1}{x} \right) dx = \int_1^e 1 dx = e - 1 \approx 1.71828$$

- Sea el siguiente algoritmo:

```

1 def eficiencia():
2     n = 10000
3
4     start = time.time()
5     suma_inv = 0
6     for _ in range(n):
7         suma_inv += generaX_inversa()
8     tiempo_inv = time.time() - start
9     media_inv = suma_inv / n
10
11    start = time.time()
12    suma_rech = 0
13    for _ in range(n):
14        suma_rech += generaX_rechazo()
15    tiempo_rech = time.time() - start
16    media_rech = suma_rech / n
17
18    print(f"Esperanza exacta: {math.e - 1:.6f}")
19    print(f"Inversa: media = {media_inv:.6f}, tiempo = {tiempo_inv:.4f} seg")
20    print(f"Rechazo: media = {media_rech:.6f}, tiempo = {tiempo_rech:.4f} seg")

```

- c) Estimación de $P(X \leq 2)$

$$P(X \leq 2) = F(2) = \ln 2 \approx 0.693147$$

- Sea el siguiente algoritmo:

```

1 def ejercicio7c():
2     def estimar_probabilidad(generator, n=10000):
3         cuenta = 0
4         for _ in range(n):
5             if generator() <= 2:
6                 cuenta += 1
7         return cuenta / n
8
9     print(f"P(X ≤ 2) exacta: {math.log(2):.6f}")
10    print(f"Inversa: {estimar_probabilidad(generaX_inversa):.6f}")
11    print(f"Rechazo: {estimar_probabilidad(generaX_rechazo):.6f}")

```

Ejercicio 8 ([Índice](#))

Enunciado:

- a) Sean U y V dos variables aleatorias uniformes en $(0, 1)$ e independientes. Pruebe que la variable $X = U + V$ tiene densidad *triangular*:

$$f(x) = \begin{cases} x & 0 \leq x < 1 \\ 2 - x & 1 \leq x < 2 \\ 0 & \text{en otro caso.} \end{cases}$$

- b) Desarrolle tres algoritmos que simules la variable X :
 - i) Usando la propiedad que X es suma de dos uniformes.
 - ii) Aplicando transformada inversa.
 - iii) Con el método de rechazo.
- c) Compare la eficiencia de los tres algoritmos. Para cada caso, estimar el valor esperado promediando 10.000 valores simulados. ¿Para qué valor x_0 se cumple que $P(X > x_0) = 0.125$?
- d) Compare la proporción de veces que el algoritmo devuelve un número mayor que x_0 con esta probabilidad.

Solución:

- a) Sean $U, V \sim \text{Uniforme}(0, 1)$ independientes.
 - La densidad conjunta es $f_{U,V}(u, v) = 1$ en el cuadrado unitario.
 - Queremos la densidad de $X = U + V$.
 - Usamos convolución:

$$f_X(x) = \int_{-\infty}^{\infty} f_U(t) f_V(x-t) dt = \int_0^1 f_V(x-t) dt, \quad 0 \leq t \leq 1$$

- Caso $0 \leq x \leq 1$:
 - Necesitamos $0 \leq t \leq 1$ y $0 \leq x-t \leq 1 \implies t \leq x$ y $t \geq x-1$ (automático pues $x-1 \leq 0$).
 - Entonces $t \in [0, x]$

- $$f_X(x) = \int_0^x 1 dt = x$$

- Caso $1 \leq x \leq 2$:
 - Necesitamos $0 \leq t \leq 1$ y $0 \leq x-t \leq 1 \implies t \geq x-1$ y $t \leq x$ (automático pues $x \leq 2$).
 - Necesitamos $t \in [x-1, 1]$

- $$f_X(x) = \int_{x-1}^1 1 dt = 1 - (x-1) = 2 - x$$

- Por lo tanto:

$$f(x) = \begin{cases} x & 0 \leq x < 1 \\ 2 - x & 1 \leq x < 2 \\ 0 & \text{en otro caso.} \end{cases}$$

- b) Tres algoritmos para simular X
 - i) Suma de dos uniformes



```

1  from random import random
2
3  def triangular_suma():
4      return random() + random()

```

- ii) Transformada inversa
 - Primero calculamos $F(x)$:

$$f(x) = \begin{cases} \frac{x^2}{2} & 0 \leq x \leq 1 \\ 2x - \frac{x^2}{2} - 1 & 1 \leq x \leq 2 \end{cases}$$

- Realizamos la inversa:
 - Si $0 \leq u \leq 0.5$:

$$u = \frac{x^2}{2} \implies x = \sqrt{2u}$$

- Si $0.5 \leq u \leq 1$:

$$\begin{aligned}
 u &= 2x - \frac{x^2}{2} - 1 \\
 x^2 - 4x + 2(u + 1) &= 0 \\
 x &= 2 \pm \sqrt{4 - 2(u + 1)} \\
 &= 2 \pm \sqrt{2 - 2u}
 \end{aligned}$$

- Para $x \in [1, 2]$ tomamos la rama negativa:

$$2 - \sqrt{2(1 - u)}$$

- Sea el siguiente algoritmo:

```

1  from random import random
2  import math
3
4  def triangular_inversa():
5      U = random()
6      if U < 0.5:
7          return math.sqrt(2 * U)
8      else:
9          return 2 + math.sqrt(2 * (1-U))

```

- iii) Método de aceptación y rechazo

- Queremos generar X con densidad f triangular en $[0, 2]$.
- Usamos $Y \sim \text{Uniforme}(0, 2)$ con densidad $g(y) = 1/2$.
- Buscamos c tal que $\frac{f(y)}{g(y)} \leq c$:

$$\frac{f(y)}{g(y)} = \frac{f(y)}{1/2} = 2f(y) \leq c$$

- El máximo de $f(y)$ es 1 (en $x = 1$), entonces $2 \cdot 1 = 2$.
- Tomamos $c = 2$.
- Luego:

$$\frac{f(y)}{c \cdot g(y)} = \frac{f(y)}{2 \cdot (1/2)} = f(y)$$

- Sea el siguiente algoritmo:



```
1  from random import random
2
3  def f_triangular(x):
4      if 0 ≤ x < 1:
5          return x
6      elif 1 ≤ x < 2:
7          return 2-x
8      else:
9          return 0
10
11 def triangular_rechazo():
12     while True:
13         Y = 2 * random()
14         U = random()
15         if U < f_triangular(Y):
16             return Y
```

- c) Comparación de eficiencia y estimación de $E[X]$
 - La esperanza exacta de la distribución triangular es:

$$E[X] = \int_0^1 x \cdot x dx + \int_1^2 x \cdot (2-x) dx = \left[\frac{x^3}{3} \right]_0^1 + \left[x^2 - \frac{x^3}{3} \right]_1^2 = \frac{1}{3} + \frac{2}{3} = 1$$

- Sea el siguiente algoritmo para comparar la eficiencia:

```

1  def eficiencia():
2      n = 10000
3      suma_sum = 0
4      suma_inv = 0
5      suma_rech = 0
6
7      start_sum = time.time()
8      for _ in range(n):
9          suma_sum += triangular_suma()
10     end_sum = time.time()
11     print(f"Suma: {end_sum - start_sum:.4f} seg, media: {suma_sum / n:.6f}")
12
13     start_inv = time.time()
14     for _ in range(n):
15         suma_inv += triangular_inversa()
16     end_inv = time.time()
17     print(f"Inversa: {end_inv - start_inv:.4f} seg, media: {suma_inv / n:.6f}")
18
19     start_rech = time.time()
20     for _ in range(n):
21         suma_rech += triangular_rechazo()
22     end_rech = time.time()
23     print(f"Rechazo: {end_rech - start_rech:.4f} seg, media: {suma_rech / n:.6f}")

```

-
- Calculo de x_0 tal que $P(X > x_0) = 0.125$

- $$P(X > x_0) = 1 - F(x_0) = 0.125 \implies F(x_0) = 0.875$$

- Como $F(1) = 0.5$ y $F(2) = 1$, buscamos en el segundo tramo:

$$F(x_0) = 2x_0 - \frac{x_0^2}{2} - 1 = 0.875$$

$$2x_0 - \frac{x_0^2}{2} = 1.875$$

- Multiplicando por 2:

$$4x_0 - x_0^2 = 3.75 \implies x_0^2 - 4x_0 + 3.75 = 0$$

- $$x_0 = 2 \pm \sqrt{4 - 3.75} = 2 \pm \sqrt{0.25} = 2 \pm 0.5$$

- La solución en $[1, 2]$ es $x_0 = 2 - 0.5 = 1.5$

- d) Comparación de la proporción $P(X > 1.5)$ con el valor teórico

- Sea el siguiente algoritmo:

```
1 def comparacion():
2     x0 = 1.5
3     p_teorica = 0.125
4     n = 10000
5
6     lista = [
7         ("Suma", triangular_suma),
8         ("Inversa", triangular_inversa),
9         ("Rechazo", triangular_rechazo)
10    ]
11
12    for nombre, func in lista:
13        cuenta = 0
14        for _ in range(n):
15            if func() > x0:
16                cuenta += 1
17        prop = cuenta / n
18        print(f"{nombre}: proporción = {prop:.4f} (teórica: {p_teorica:.3f})")
```

Ejercicio 9 ([Índice](#))

Enunciado:

Solución:

Ejercicio 10 ([Índice](#))

Enunciado:

Solución:

Ejercicio 11 ([Índice](#))

Enunciado:

Solución:

Ejercicio 12 ([Índice](#))

Enunciado:

Solución:

Ejercicio 13 ([Índice](#))

Enunciado:

Solución:

Ejercicio 14 ([Índice](#))

Enunciado:

Solución:
